# INTRODUCTION À LARAVEL 12 — COMPRENDRE, INSTALLER ET UTILISER LES BASES

Par Robert DIASSÉ

# 1. Introduction : Pourquoi Laravel ?

Laravel est un framework backend moderne construit en PHP orienté objet.

Son objectif :

- faciliter le développement,
- organiser le code,
- accélérer les projets professionnels,
- fournir des outils prêts à l'emploi (authentification, API, base de données, sécurité…).

Avant de programmer avec Laravel, il est essentiel de comprendre comment il fonctionne, comment il s'installe et comment son architecture interne est structurée.

Ce document guide pas à pas pour démarrer correctement.

---

# 2. Pré-requis techniques indispensables

Laravel 12 exige :

- **PHP ≥ 8.2**
- **Composer** (gestionnaire de dépendances PHP)
- **Extensions PHP activées :**
  `pdo`, `pdo_mysql` ou `pdo_sqlite`, `openssl`, `mbstring`, `fileinfo`, `ctype`, `tokenizer`.

---

## 2.1 Vérifier la version PHP

```
php -v
```

Si la version affichée est **< 8.2**, il faut installer une version plus récente de XAMPP/WAMP.

---

### 2.2 Vérifier et activer les extensions PHP

Sur Windows + XAMPP :

1. Ouvrir :
   `C:\xampp\php\php.ini`
2. Chercher une extension, exemple :

   ```
   ;extension=pdo_mysql
   ```

3. Retirer le `;`
4. Enregistrer
5. Redémarrer Apache

Vérifier :

```
php -m
```

---

# 3. Installer Composer

Composer sert à :

- installer Laravel,
- installer des bibliothèques,
- gérer les mises à jour,
- charger automatiquement les classes.

Vérifier Composer :

```
composer -V
```

---

# 4. Installer Laravel 12

Créer un nouveau projet :

```
composer create-project laravel/laravel mon_projet
```

Entrer dans le projet :

```
cd mon_projet
```

Démarrer le serveur :

```
php artisan serve
```

Le site doit s'afficher ici :
http://127.0.0.1:8000

---

# 5. Architecture Laravel : Structure complète générée

Quand Laravel est installé, voici **la structure exacte** qu'on obtient :

```
mon_projet/
│
├── app/
│   ├── Console/
│   ├── Exceptions/
│   ├── Http/
│   │   ├── Controllers/
│   │   ├── Middleware/
│   ├── Models/
│
├── bootstrap/
│
├── config/
│
├── database/
│   ├── factories/
│   ├── migrations/
│   ├── seeders/
│
├── public/
│   ├── index.php
│
├── resources/
│   ├── views/
│   ├── js/
│   ├── css/
│
├── routes/
│   ├── web.php
│   ├── api.php
│
```

```
├── storage/
│
├── tests/
│
├── vendor/
│
├── .env
└── composer.json
```

Maintenant, voyons **à quoi sert chaque dossier**.

---

# 6. Explication claire des dossiers Laravel

## 6.1 Le dossier **app/**

C'est le cœur de l'application.
Il contient tout le code que vous écrivez vous-même.

### Ce qu'on y trouve :

- **Http/Controllers/** → traite les requêtes du navigateur ou d'une API
- **Models/** → représente les tables de la base de données
- **Middleware/** → intercepte et filtre les requêtes (sécurité, sessions…)

**C'est ici que réside la logique métier de l'application.**

---

## 6.2 Le dossier **routes/**

Il contient les fichiers qui définissent les URL de votre application.

- `web.php` → routes web (retournent des vues HTML)
- `api.php` → routes API REST (retournent du JSON)

Dans Laravel, une route = une URL + une action.

Exemple simple :

```php
Route::get('/hello', function () {
    return "Bonjour !";
});
```

---

## 6.3 Le dossier **resources/**

Il contient tout ce qui concerne l'interface (front-end) :

- **views/** → fichiers Blade (HTML amélioré)

- **js/** → scripts
- **css/** → styles

---

## 6.4 Le dossier **database/**

Contient tous les éléments liés à la base de données :

- **migrations/**
  Scripts pour créer/modifier les tables.
- **seeders/**
  Pour insérer des données initiales (admins, rôles…)
- **factories/**
  Pour générer de fausses données automatiques lors des tests.

---

## 6.5 Le dossier **public/**

Dossier exposé sur Internet.

Il contient :

- **index.php** → point d'entrée du site
- fichiers CSS/JS/images compilés.

---

## 6.6 Le fichier **.env**

Contient toutes les configurations sensibles :

Exemple :

```
APP_NAME=Laravel
APP_ENV=local
DB_DATABASE=laravel
DB_USERNAME=root
```

---

## 6.7 Le fichier **composer.json**

Décrit les dépendances utilisées par Laravel.

Exemple :

```
"require": {
    "php": "^8.2",
    "laravel/framework": "^12.0"
}
```

Chaque fois qu'on installe une bibliothèque, elle apparaît ici.

---

# 7. Artisan : l'outil en ligne de commande de Laravel

Artisan est un outil extrêmement puissant fourni avec Laravel.

Il permet :

- créer des controllers
- créer des modèles
- créer des migrations
- vider le cache
- lancer le serveur
- exécuter les migrations
- générer les clés d'application

### Quelques commandes essentielles :

Démarrer le serveur :

```
php artisan serve
```

Créer un controller :

```
php artisan make:controller ServiceController
```

Créer un modèle :

```
php artisan make:model Service
```

Créer une migration :

```
php artisan make:migration create_services_table
```

Artisan automatise ce qu'on ferait manuellement.

---

# 8. Blade : le moteur de template Laravel

Blade est un outil qui permet d'écrire du HTML **avec des fonctionnalités supplémentaires**.

Exemple simple :

```
<h1>{{ $title }}</h1>
```

Les `{{ ... }}` affichent des variables PHP de façon sécurisée.

Exemple de template avec conditions :

```
@if($age >= 18)
    <p>Adulte</p>
@else
    <p>Mineur</p>
@endif
```

### Avantages de Blade :

- syntaxe simple,
- évite de mélanger trop de PHP dans les vues,
- permet les layouts (modèles de pages),
- permet de séparer HTML et logique.

---

# 9. Eloquent ORM : interaction avec la base de données

ORM = **Object Relational Mapping**

Cela signifie :

- une table → un modèle
- une ligne → un objet
- une colonne → une propriété

Exemple :
La table **users** est représentée par le modèle **User**.

Exemple de récupération de données :

```
$users = User::all();
```

Créer un utilisateur :

```
User::create([
    'name' => 'Jean',
```

```
    'email' => 'jean@test.com'
]);
```

Mis à jour :

```
$user->update([...]);
```

Supprimer :

```
$user->delete();
```

Eloquent simplifie énormément l'accès à la base.

---

# 10. Tester l'installation : première route + première vue

## 10.1 Première route

Dans `routes/web.php` :

```
Route::get('/', function () {
    return view('home');
});
```

---

## 10.2 Créer la vue

Créer `resources/views/home.blade.php` :

```
<h1>Bienvenue dans Laravel</h1>
<p>Ceci est votre première page.</p>
```

Afficher :
http://127.0.0.1:8000/

---

# 11. Créer un layout (modèle principal)

Créer le layout :
`resources/views/layouts/app.blade.php`

```
<!DOCTYPE html>
<html>
<head>
    <title>@yield('title')</title>
</head>
<body>

<header>
    <h2>Application Laravel</h2>
</header>

<main>
    @yield('content')
</main>

</body>
</html>
```

---

### 11.1 Utiliser le layout

Modifier `resources/views/home.blade.php` :

```
@extends('layouts.app')

@section('title', 'Accueil')

@section('content')
    <h1>Bienvenue !</h1>
    <p>Voici votre première vue avec un layout.</p>
@endsection
```

---

# 12. Vous êtes maintenant prêt pour le projet fil rouge

Ce document vous permet :

- de comprendre l'architecture Laravel,
- de faire une installation propre,
- d'activer les extensions,
- d'utiliser Artisan,
- de créer vos premières routes et vues,
- de comprendre Blade et Eloquent ORM.